

Python & Scikit-learn Practice Session

Working notes from a hands-on session covering CSV data processing without pandas, and a full scikit-learn churn classification workflow.

Python

csv module

scikit-learn

Classification

01

Python — CSV Data Processing

Reading, filtering, grouping & writing CSV data

02

Scikit-learn — Classification

Churn prediction with Logistic Regression

03

Interview Q&A Recap

Key questions practiced from this session

Python — CSV Data Processing

Built-in csv module · no pandas

1 Reading CSV without Pandas

Used Python's built-in `csv` module:

```
import csv

def read_employees(filename):
    employees_list = []

    with open(filename, "r") as f:
        reader = csv.DictReader(f)

        for row in reader:
            employees_list.append(row)

    return employees_list
```

IMPORTANT

`csv.DictReader()` converts each row into a dictionary, e.g. `{"id": "101", "name": "Alice", "department": "IT", "salary": "75000"}`. CSV values are initially strings — so `float(employee["salary"])` is required for numerical calculations.

2 Calculate Average Salary

```
def avg_salary(employees):
    total_salary = 0

    for employee in employees:
        salary = float(employee["salary"])
        total_salary += salary

    return total_salary / len(employees)
```

RESULT

For our dataset: average salary = `70000`

3 Filter Employees

Initially, a function returning only names:

```
def high_salary_employees(employees, average):
    result = []

    for employee in employees:
        salary = float(employee["salary"])

        if salary > average:
            result.append(employee["name"])

    return result
```

OUTPUT

`['Alice', 'Charlie', 'Eva']`

Then a version returning complete dictionaries — useful when writing back to CSV:

```
def get_high_salary_employees(employees, average):
    result = []

    for employee in employees:
        salary = float(employee["salary"])

        if salary > average:
            result.append(employee)

    return result
```

4 Group Data by Department

KEY PATTERN

Important dictionary pattern for grouping:

```
if department not in department_salaries:
    department_salaries[department] = []

department_salaries[department].append(salary)
```

Complete function:

```
def grp_by_dept(employees):
    department_salaries = {}

    for employee in employees:
        department = employee["department"]
        salary = float(employee["salary"])

        if department not in department_salaries:
            department_salaries[department] = []

        department_salaries[department].append(salary)

    return department_salaries
```

RESULT

```
{"IT": [75000, 90000, 85000], "HR": [55000, 60000], "Finance": [65000]}
```

5 Dictionary `.items()`

```
for dept, salaries in departments.items():
```

`dept` → dictionary key | `salaries` → dictionary value

Example: `dept = "IT"`, `salaries = [75000, 90000, 85000]`

Loop form	Iterates over
<code>for x in dictionary:</code>	keys
<code>for x in dictionary.values():</code>	values
<code>for key, value in dictionary.items():</code>	both

6 Department-wise Average

```
def avg_salary_dept(salaries):
    return sum(salaries) / len(salaries)
```

```
def avg_salary_by_dept(departments):
    department_salaries = {}

    for dept, salaries in departments.items():
        average = avg_salary_dept(salaries)
        department_salaries[dept] = average

    return department_salaries
```

Used: `avg_salary_by_dept(grp_by_dept(employees))`

Department	Average Salary
IT	83,333.33
HR	57,500
Finance	65,000

7 Writing CSV

Used `csv.DictWriter` :

```
with open("high_salary_employees.csv", "w", newline="") as f:
    fieldnames = ["id", "name", "department", "salary"]

    writer = csv.DictWriter(f, fieldnames=fieldnames)

    writer.writeheader()

    for employee in high_salary:
        writer.writerow(employee)
```

IMPORTANT

`writer.writeheader()` → writes column names. `writer.writerow(employee)` → writes one employee/row.

8 Large CSV Insight

KEY INSIGHT

For a very large CSV (e.g. 1 million rows), process records one by one rather than loading the entire dataset into memory.

Instead of: `employees = read_employees("employees.csv")`

```
with open("employees.csv", "r") as f:
    reader = csv.DictReader(f)

    for employee in reader:
        # process one row
        ...
```

INTERVIEW ANSWER

For a very large CSV, I would process records incrementally to reduce memory usage.

Scikit-learn — Classification

Customer churn prediction

Problem: built a customer churn classification model.

Features: Age, MonthlyCharges, Tenure | **Target:** Churn

1 Dataset Structure

```
X = np.array([
    [22, 45, 2],
    [35, 60, 12],
    ...
])

y = np.array([
    1, 0, 1, 0, ...
])
```

`X` → features/input | `y` → target/output

2 Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Training data → Model learns
Testing data → Evaluate on unseen data

INTERVIEW ANSWER

We keep some data unseen during training so we can evaluate how the trained model performs on unseen data.

3 Logistic Regression

```
model = LogisticRegression()

model.fit(X_train, y_train)
```

`fit()` learns the patterns/parameters from the training data.

4 Prediction

```
y_pred = model.predict(X_test)
```

`predict()` uses the trained model to generate predictions for input data.

5 Evaluation Metrics

Practiced: `accuracy_score()`, `precision_score()`, `recall_score()`, `f1_score()`, `confusion_matrix()`

KEY INTERVIEW POINT

Don't judge a classification model using accuracy alone.

Metric	Value
Accuracy	95%
Precision	70%
Recall	40%

For a churn problem, this may not be good — 40% recall means the model is missing many actual churners.

6 New Prediction

```
new_customer = np.array([[30, 82, 6]])

prediction = model.predict(new_customer)
```

Complete practical flow:

Dataset → X / y → Train/Test Split → Logistic Regression
→ fit() → predict() → Evaluation → New Prediction

Interview Questions Practiced

Recap of Q&A from this session

fit() vs predict() — what's the difference?

fit: used to learn from training data. **predict:** used to generate predictions using the trained model.

Why do a train/test split?

To evaluate the model on data it has not seen during training.

Why isn't accuracy always enough?

Because accuracy can hide poor performance on the class we actually care about. For problems like churn, precision and recall can be more informative.